

Experiment 3 - Sequential Synthesis and Post Simulation

Ali Dehghani,
810103414

Abstract— This report details the Verilog HDL design, synthesis, and simulation of a digital serial transmitter system. The architecture integrates a Moore-type sequence detector to identify a “1001” pattern, a synchronous counter to maintain a transmission window for 9 clock cycles, and a one-pulser circuit for reliable physical input handling. Both pre-synthesis behavioral analysis and post-synthesis timing simulations—incorporating Standard Delay Format (SDF) files—were conducted in ModelSim to verify the system’s functionality and timing reliability on an Intel Cyclone IV E FPGA.

Keywords— Verilog HDL, FPGA, Sequence Detector, Finite State Machine (FSM), Serial Transmitter, Post-Synthesis Simulation.

I. INTRODUCTION

This experiment focuses on the design, synthesis, and hardware-level simulation of sequential logic circuits using Verilog HDL. The primary objective is to construct a serial transmitter system consisting of a Moore finite state machine (FSM) for pattern detection, a synchronous 4-bit counter, and a one-pulser module to interface with physical push-buttons. Furthermore, this study bridges the gap between ideal logical behavior and real-world hardware by utilizing Intel Quartus for synthesis targeting a Cyclone IV E FPGA. By performing post-synthesis simulations in ModelSim with annotated gate and routing delays, the experiment provides a comprehensive methodology for verifying circuit timing and ensuring robust functionality under realistic physical conditions.

II. SERIAL TRANSMITTER

A. Onepulser

1) Verilog description:

For implementing the delay of that and also use the clkPB I used 3 flip flop first one work based on clkPB value and second one get the value of the first one, third one used for the one clock cycle delay and I assigned the output to logic and of second one and the third contradiction(for delay support and also only one time high pulse each time the clkPB pushed).

2) Test the Design in Modelsim:

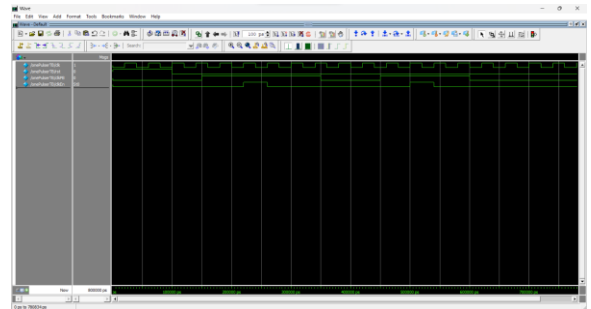


Fig. 1 Waveform of onepulser test

As we can see from Fig. 1 every time we make clkPB 1 the clkEn gets 1 only one time and for one clock cycle, so it's work correctly.

B. Orthogonal Finite State Machine

1) State Diagram of the Sequence Detector:



Fig. 2 State diagram of the sequence detector

2) Verilog Description of the Sequence Detector:

I implement it based on the state diagram and also considered all it's output in Fig. 1 of lab description except serOut because it is a moore machine and I initialize it inside top module.

3) Verilog Description of the Counter:

A simple 4 bit counter that only count from 0 to 8 and them become zero.

4) Top-level Verilog description of the Transmitter Circuit:

Take instance from two previous modules and also initialize serOut to serIn or zero based on the serOutValid of moore machine.

5) Testbench for the Serial Transmitter Unit:

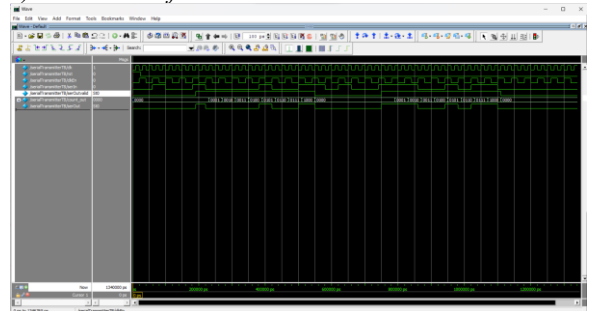


Fig. 3 Serial transmitter waveform

As we can see from Fig. 2 after the moore machine reach to S4 state the serOutValid is 1 for 9 clock cycle also the value of serOut in these cycles is equal to serIn. Also the counter work and reset correctly.

C. Seven Segment Display

According to the lab, the Seven-Segment Display (SSD) module is intended for hardware implementation on the FPGA to display the number of valid outputs. The module takes a 4-bit binary input from the counter and decodes it into a 7-bit Active-Low signal to drive the segments.

Since the counter module only counts from 0 to 9, its 4-bit output is intrinsically equivalent to a valid BCD representation. Therefore, no additional binary-to-BCD conversion logic is required. The 4-bit output is directly fed into the SSD module, which successfully maps the values 0 – 9 to their corresponding hexadecimal/decimal visual patterns on the display.

III. PRE-SYNTHESIS SIMULATION

A. Provide All Verilog Codes

Put onePulser and serial transmitter inside a folder and create a modelsim project.

B. Testbench

In the testbench, the clock was configured to operate at the required frequency of 20 MHz, which corresponds to a 50 ns period; therefore, the clock signal was toggled every 25 ns. The push-button input (clkPB) was simulated using pulses longer than one clock cycle (for example, remaining high for 200 ns) to verify that the OnePulser module correctly converts each long pulse into a single clock-cycle pulse. Additionally, the serIn input sequence was generated such that the pattern 1001 appears twice during the simulation. After each detection of 1001, a delay of nine clock cycles (with either idle or random data) was introduced to allow the counter to complete its operation before applying the second occurrence of the sequence.

C. Simulation

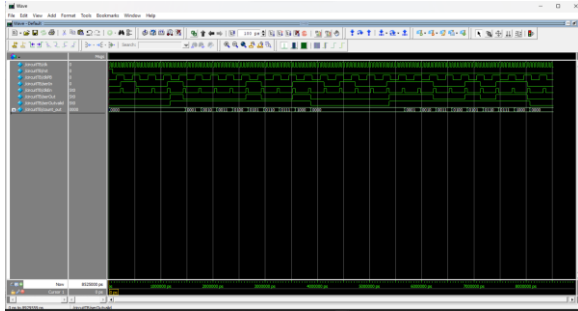


Fig. 4 Circuit waveform

D. Report the Result of Pre-synthesis Simulation

As we can see in the waveform circuit is working correctly, counter goes up to 9 from 0 and then reset to 0 (when the serOutValid becomes 1).

IV. POST-SYNTHESIS SIMULATION

A. Testbench Design

In the post-synthesis simulation, since the clkEn signal is an internal node and might be renamed, modified, or optimized away by the synthesis tool, we cannot rely on the

@(posedge clkEn) statement. Instead, a task is utilized to accurately simulate the physical behavior of pressing the push-button and applying the input data over time.

B. Simulation Results and Analysis:

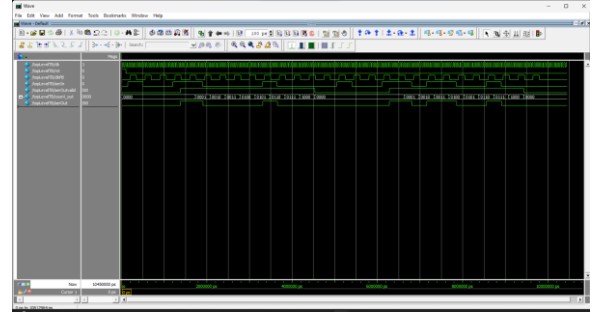


Fig. 5 TopLevel module waveform

In this section, the top-level module integrating the onePulser and serialTransmitter was synthesized in Quartus. Following synthesis, the generated netlist and SDF file were used in ModelSim to observe the behavior of the circuit with realistic gate and routing delays.

The post-synthesis simulation results successfully demonstrate the detection of the 1001 sequence twice. For both detections:

The sequence detector correctly identified the 1001 pattern on the serIn line.

Upon detection, the serOutvalid signal was successfully asserted to logic 1.

The counter reliably counted for exactly 9 active cycles while serOutvalid remained high, routing serIn to serOut.

The waveforms confirm that the circuit functions as expected under real hardware conditions, and the applied delays did not cause any logical failures or timing violations during the two consecutive sequence detections.